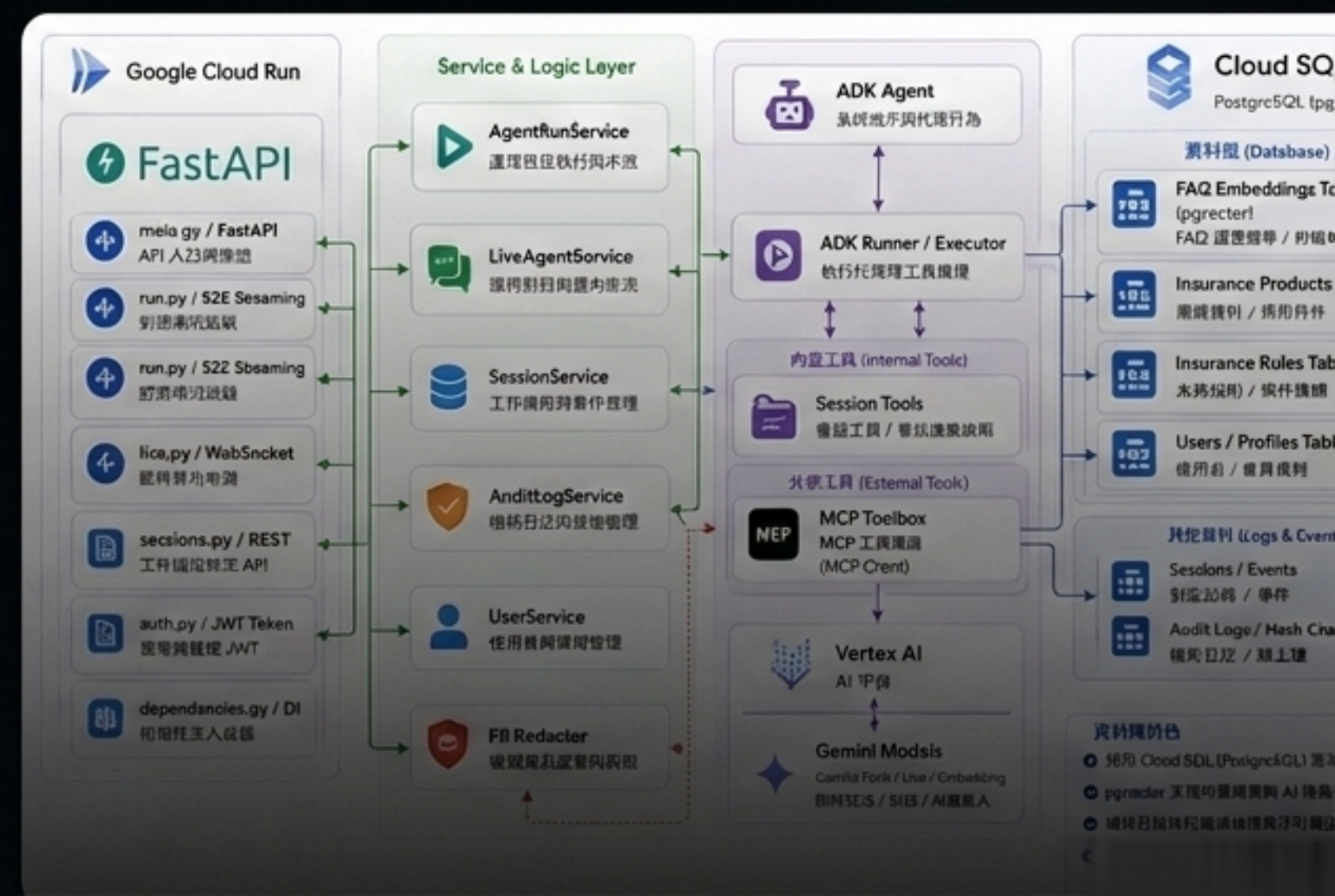


Reference Architecture / Living Artifact

Google ADK即時語音串流 實現GCP平台與DevOps 的整合應用

Google ADK + MCP Toolbox 企業級端到端
Agent 架構實戰。



重塑 Agent 的工程標準

Demo 聊天機器人

資料存取



自由撰寫 SQL 的風險



上下文



依賴無結構的對話歷史
(Chat History)

互動模式



單一文字問答

品質確保



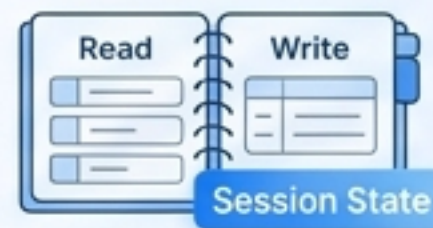
依賴 Prompt 感覺
(Vibes)

Production-Ready Agent

受控的 MCP Tools
查詢邊界



讀寫明確的結構化狀態
(Session State)



SSE 思考過程串流 /
WebSocket 全雙工多模態



Audit Hash Chain 與
ADK Evalsets 量化驗證



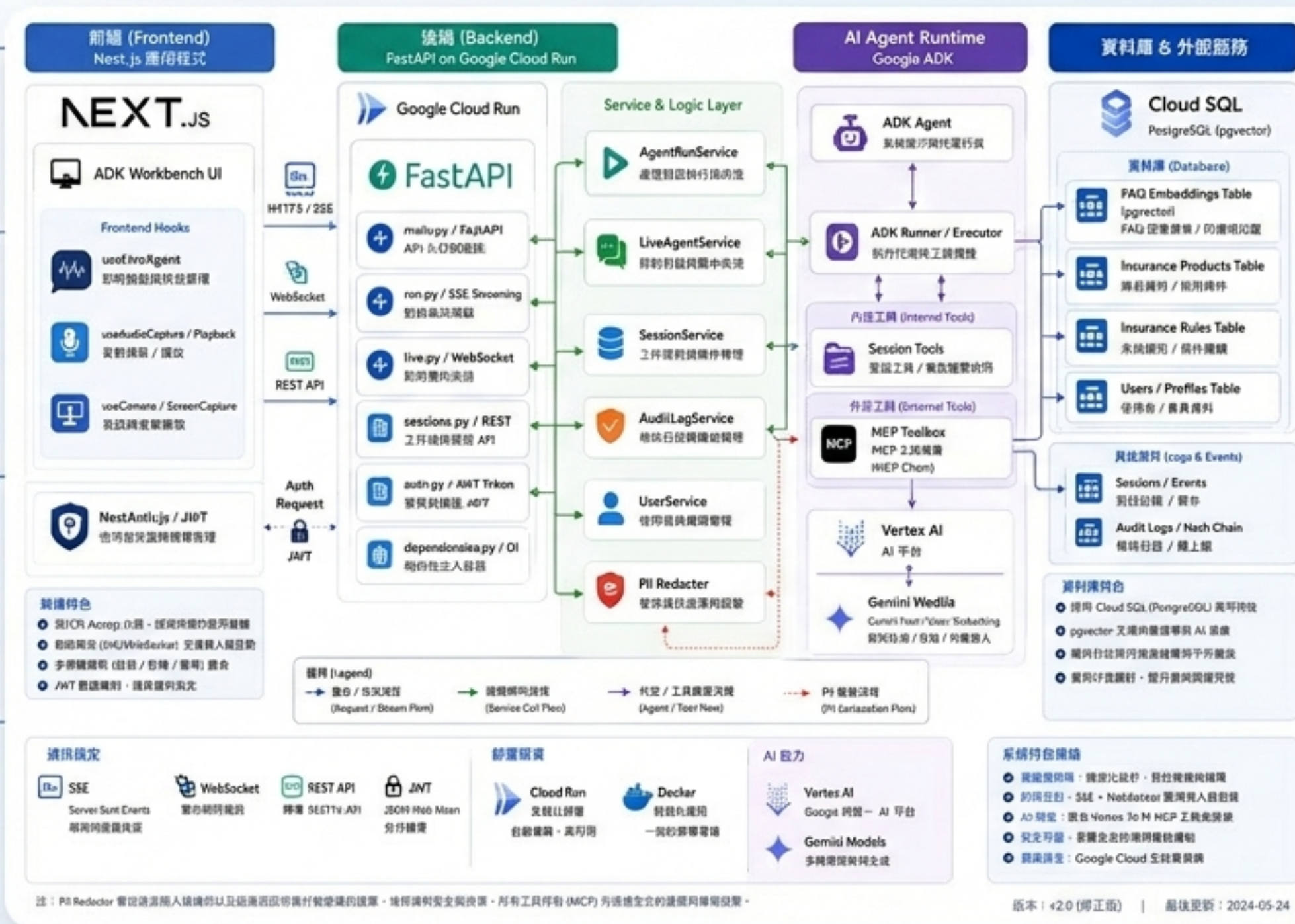
分層架構：一個完整 Agent App 的骨架

前端串流 (Next.js)：承載 SSE 與 WebRTC 多模態互動介面。

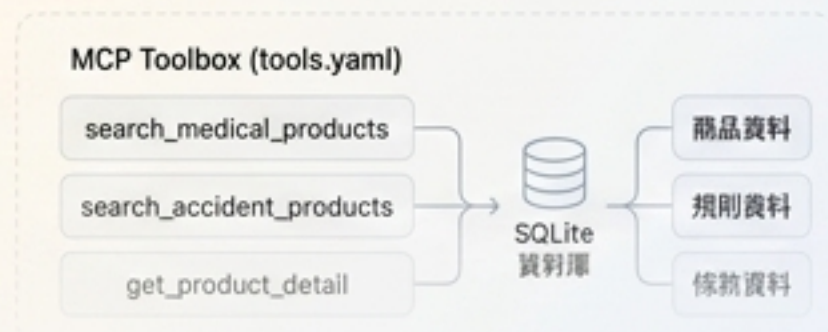
後端調度 (FastAPI)：處理 REST, WebSocket, 與 Auth/JWT 驗證。

代理核心 (Google ADK)：封裝 LLM 決策邏輯、工具選擇與狀態管理。

知識與資料 (PostgreSQL/pgvector)：結構化商品庫與 FAQ 語意向量雙引擎。



可控流程 × 清晰邊界 × 可追溯



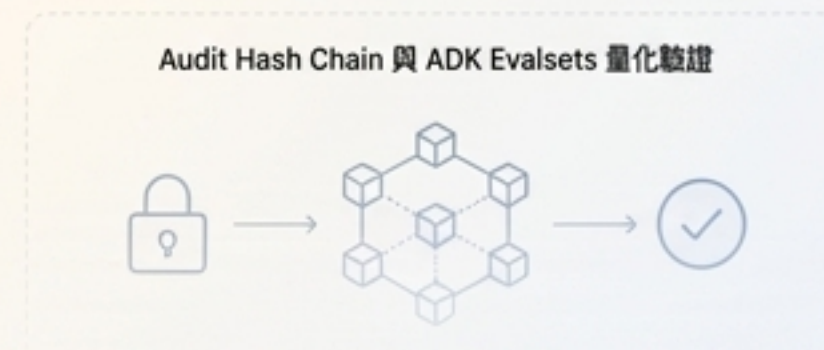
業務邏輯前置

透過 MCP 工具，將保險推薦規則與預算可行性（budget_fit）前置於查詢層，拒絕模型幻覺。



即時多模態

支援 Voice/Vision 串流，突破文字限制，打造全雙工的 Live Agent 互動。



安全與合規

內建 PII 自動脫敏、JWT 權限管控與 Audit Hash Chain 防竄改機制。

Agent 核心：決策的封裝與調度

Agent 的可維護性來自於架構的解耦。透過 Factory Pattern 組裝 Google ADK，將模型、工具與提示詞分離。

- **System Prompt**：定義保險顧問的 Persona 與基礎行為準則。
- **Local Session Tools**：讀寫使用者狀態與對話記憶。
- **MCP Toolbox**：介接外部關聯式資料與向量資料庫。



MCP Toolbox：資料查詢的受控邊界

賦予 LLM 查資料的能力，但不賦予自由撰寫 SQL 的風險。



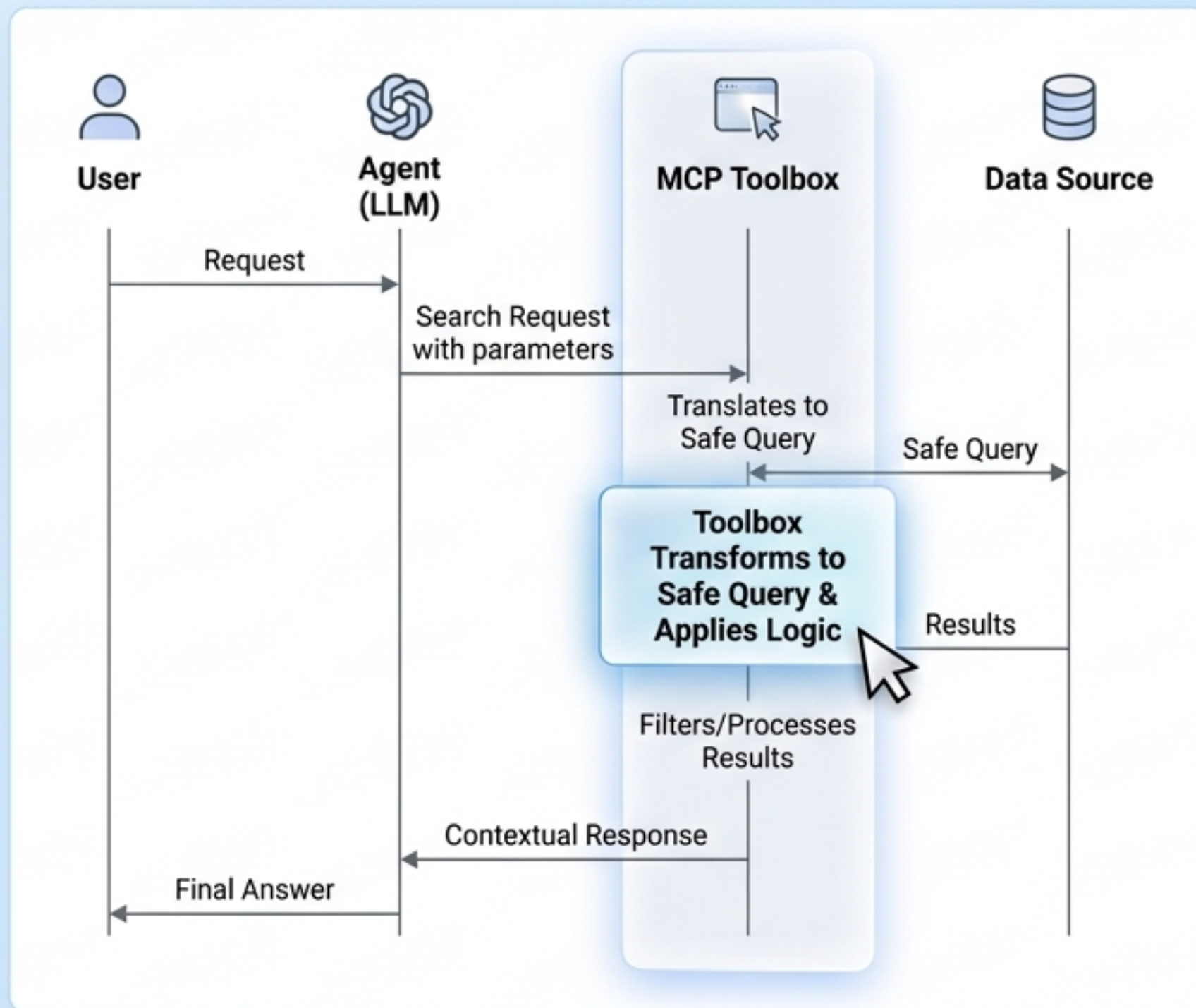
參數化查詢

模型僅能輸入年齡、預算等變數，由 Toolox 轉化為安全查詢。



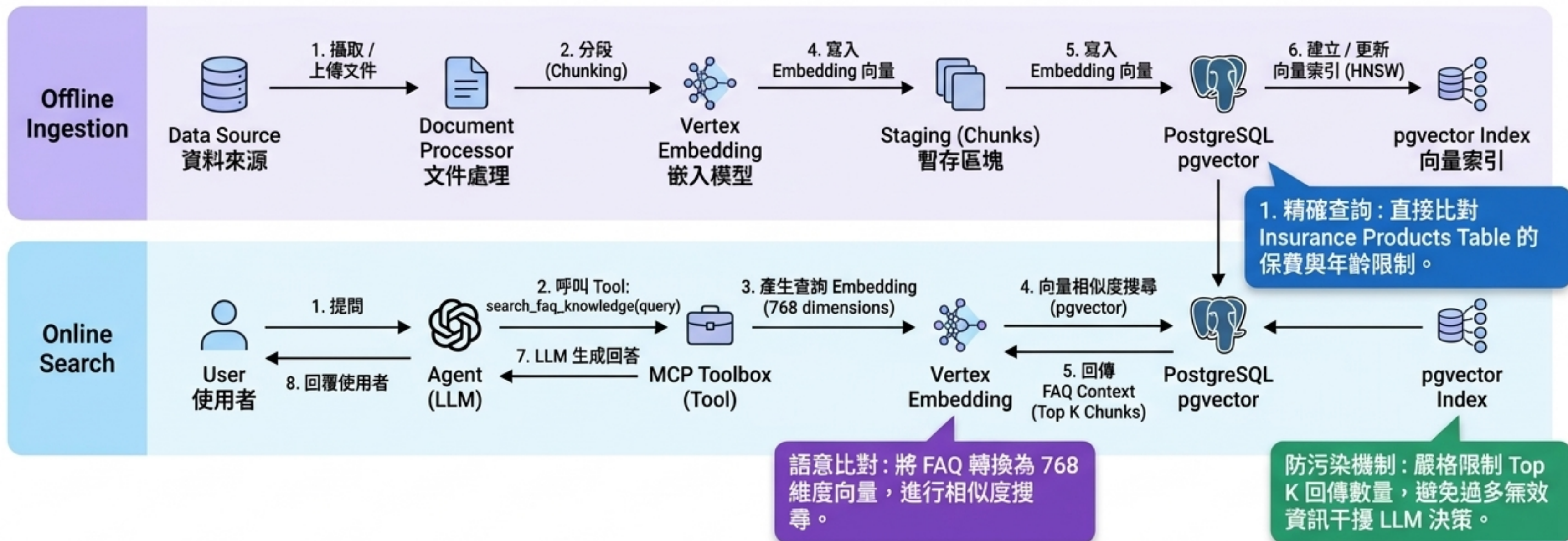
邏輯攔截

將「預算是足夠 (budget_fit)」等精確業務規則寫死在工具層，而非依賴模型自行運算。



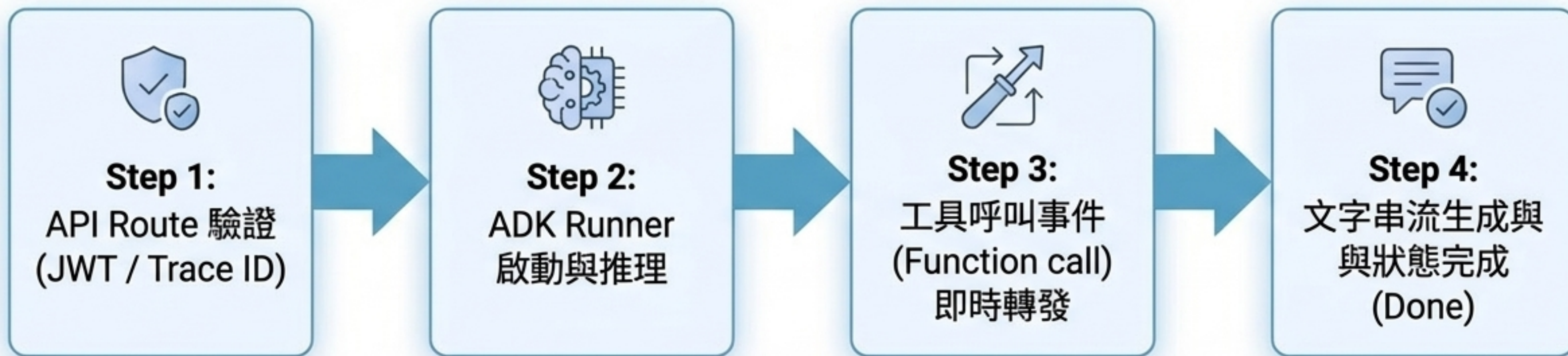
結構化資料與語意檢索的融合

PostgreSQL + pgvector 雙引擎架構。同時滿足精確的保單條件查詢與模糊的條款語意解釋。



SSE 串流：讓思考過程可視化

Agent 的執行不該是一個漫長的等待黑箱。我們將 ADK Event 映射為前端可讀的 Timeline Envelope。



Live Agent：全雙工的多模態互動

Bidirectional WebSocket / Multimodal

突破傳統 HTTP 的問答限制。透過 WebSocket 處理音訊、影像與文字的並行調度。

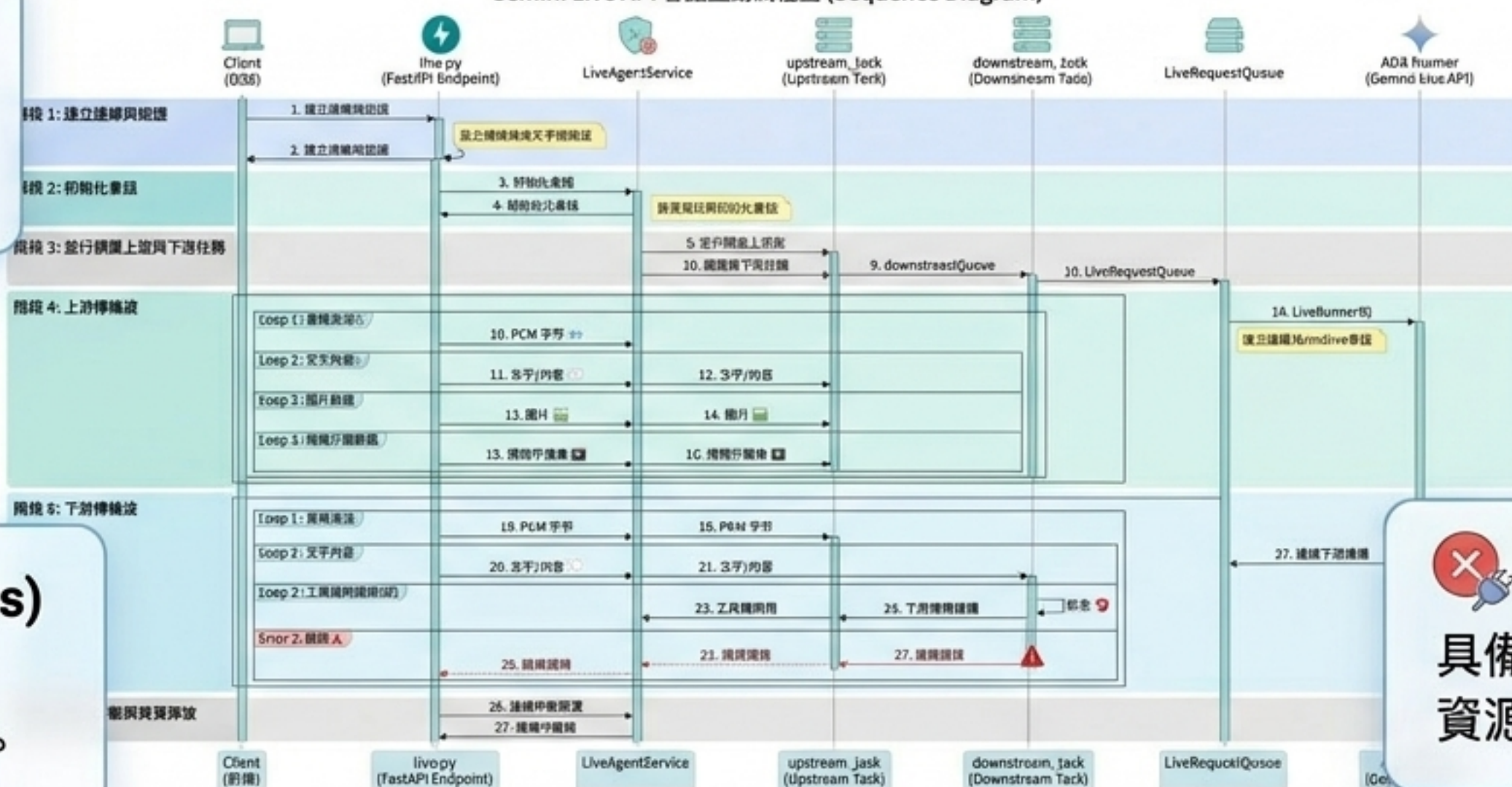
資源管理

圖片自動縮放轉 JPEG
降低處理延遲。

並行佇列 (Queues)

獨立的 Upstream 與
Downstream 任務分離。

Gemini Live API 會話互動流程圖 (Sequence Diagram)

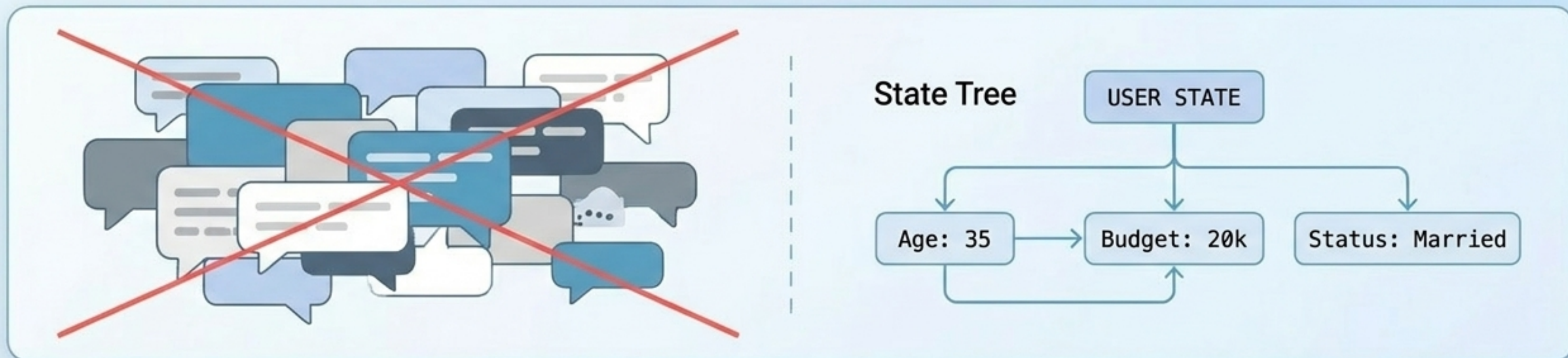


中斷與錯誤

具備即時連線中斷與
資源釋放機制。

狀態感知：記憶是結構化的 Data

揚棄隱性、易丟失的對話歷史。透過 Session Tools 讓 Agent 主動讀寫明確的使用者狀態。



主動更新：

當使用者提供新資訊，Agent 呼叫 `save\_user\_profile` 更新狀態樹。

精準追問：

發現狀態缺失（如未告知預算），觸發系統追問機制。

上下文連貫：

記憶「上次推薦商品」，確保後續詢問（如等待期）能無縫接軌。

信任基礎：三道安全防線

Agent 的自主能力越強，系統的稽核邊界就必須越清晰。



1. 權限認證 (Auth)

NextAuth 保護前端，後端
依賴 JWT 驗證 API 與
WebSocket 連線。



2. 隱私脫敏 (PII)

在寫入 Log 或傳送給 LLM
前，自動遮蔽身分證、電
話等敏感個資。



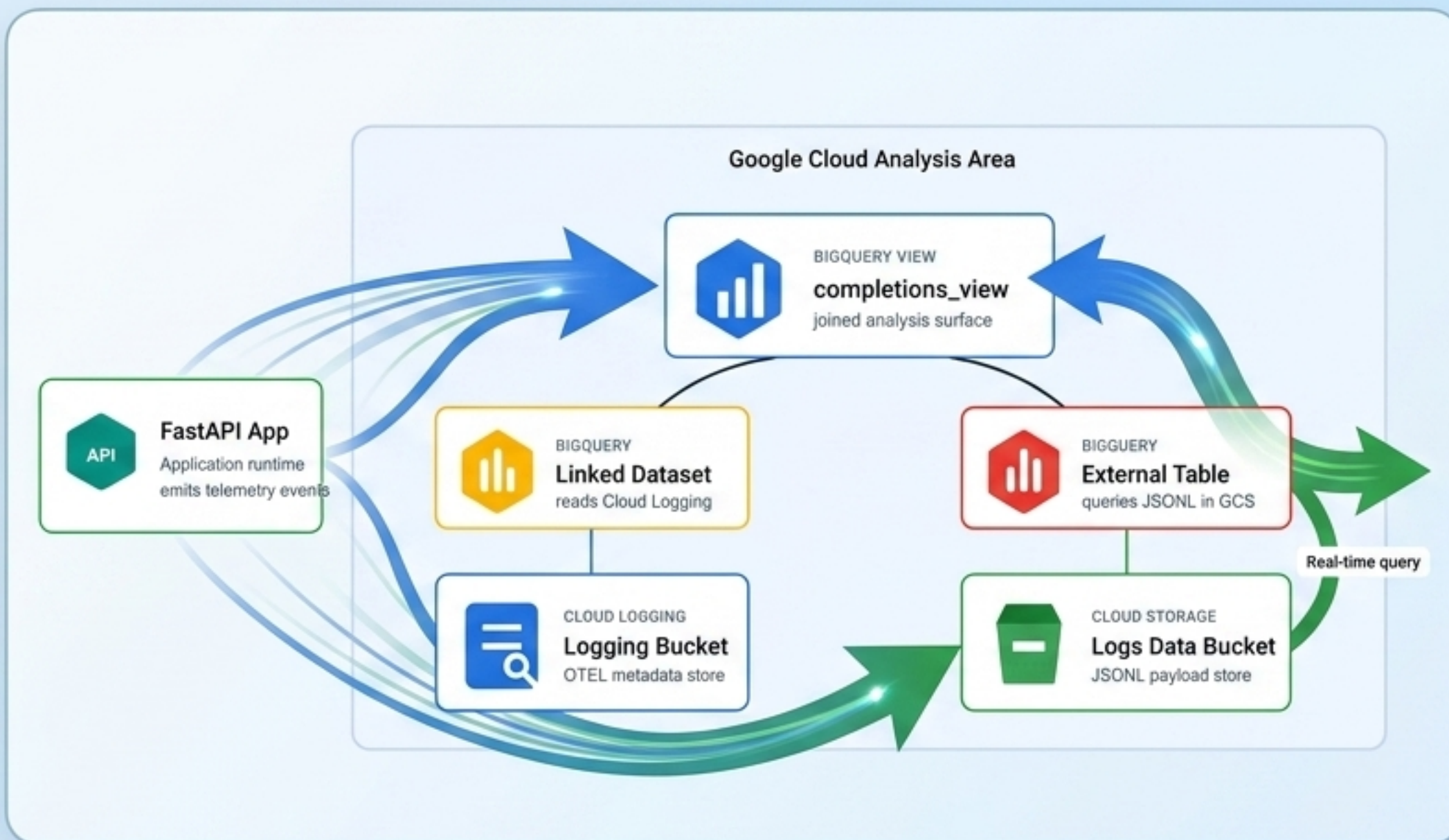
3. 稽核追蹤 (Audit)

每一筆 Agent 決策透過
Prev_Hash + Event_Hash
形成 Audit Hash Chain，
確保紀錄防竄改。

可觀測性：超越 Log 的營運視角

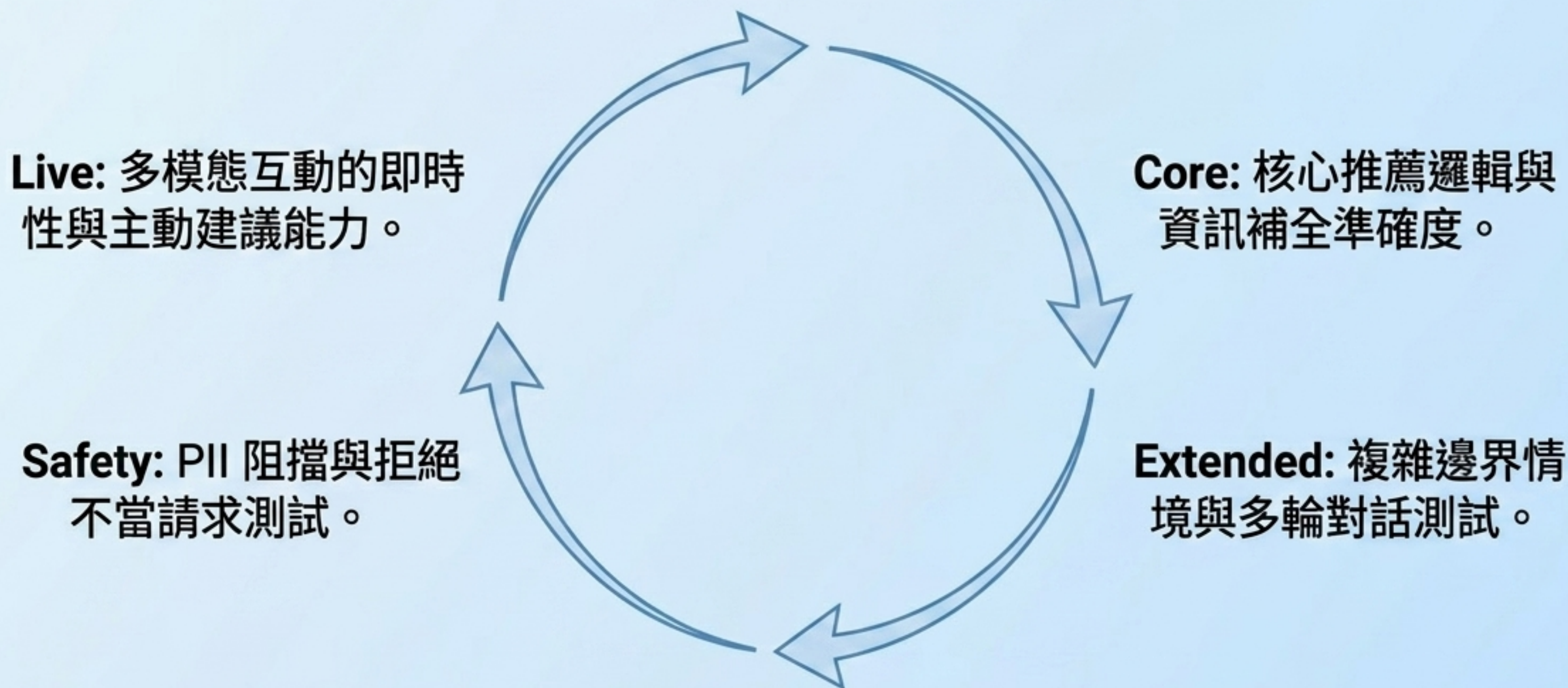
結合 OpenTelemetry 與 BigQuery Agent Analytics，看見 Agent 生命週期的全貌。

- ✓ Token 消耗與運行成本
- ✓ MCP 工具呼叫延遲與成功率
- ✓ Session 狀態變更軌跡
- ✓ JSONL 格式的備份與重放分析



評測驅動：從憑感覺到量化驗證

Prompt 的每一次修改都必須經過系統化驗證。Agent 的品質是持續評測的工程結果。



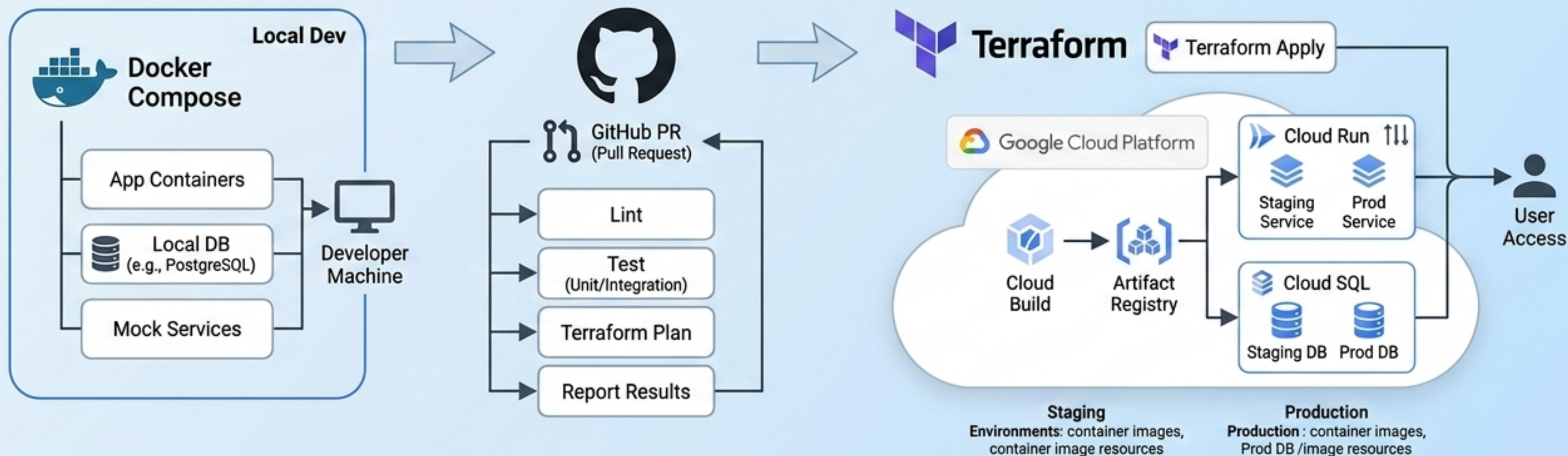
落地路徑：從本機開發到雲端佈署

基礎設施即程式碼 (IaC)，確保從 Dev 到 Prod 的環境一致性。

Day 0: Docker Compose
本機環境快速啟動。

CI/CD: GitHub PR 檢查
(Lint/Test/Plan)。

Staging/Prod: 透過 Terraform 自動部署與擴展
Google Cloud Run 與 Cloud SQL。



結論：Agent 工程的本質

業務邏輯必須前置於工具，絕不依賴模型自行推演，從根本降低幻覺。

結構化狀態管理 (State) 必須取代無邊界的對話歷史 (Chat History)。

沒有完整的可觀測性 (Telemetry) 與量化評測 (Evals)，Agent 就不具備上線條件。